

Conversões de C para MIPS

① `int maximo (int a, int b)`
 `if (a > b)`
 `return a;`
 `}`
 `return b;`
 `}`

`$a0 = a`

`$a1 = b`

maximo:

`slt $t0, $a, $a1`

`seq $t0, $zero, Rtb`

`add $v0, $a0, $zero`

`jr $ra`

Rtb:

`add $v0, $a1, $zero`

`jr $ra`

② `int soma-array (int *A, int n)`
 `int s = 0;`
 `int i = 0;`
 `while (i < n)`
 `s += A[i];`
 `i++;`
 `}`
 `return s;`
 `}`

soma-array:

`li $v0, 0 → s`

`$v1 = n`

`li $t0, 0 → i`

Loop:

`bge $t0, $v1, EndLoop`

`sll $t1, $t0, 2`

`add $t1, $t1, $t0`

`lw $t2, 0($t1)`

`add $v0, $v0, $t2`

`addi $t0, $t0, 1`

`j Loop`

`EndLoop`

`jr $ra`

③ `int count_pos (int *v, int n)`
`int cnt = 0;`
`int i = 0;`
`while (i < n)`
`if (v[i] > 0)`
`cnt++;`
`}`
`i++;`
`}`
`return cnt;`
`}`

Count_pos:

li \$v0, 0 $\rightarrow$ cnt \$v1 $\rightarrow$ n

li \$t0, 0 $\rightarrow$ i

Loop:

lge \$t0, \$v1, EndLoop

sl \$t1, \$t0, 2

add \$t1, \$t1, \$t0

lw \$t2, 0(\$t1)

sll \$t3, \$zero, \$t2

add \$v0, \$v0, 1

addi \$t0, \$t0, 1

j Loop

EndLoop:

jr \$ra

④ `void double (int *A, int n)`
`int i = 0;`
`while (i < n)`
`A[i] = A[i] * 2;`
`i++;`
`}`
`}`

double:

li \$t0, 0 $\rightarrow$ i

$S_0 = A$

$S_1 = n$

Loop:

lge \$t0, \$s1, EndLoop

sl \$t1, \$t0, 2

add \$t1, \$t1, \$s0

lw \$t2, 0(\$t1)

sll \$t2, \$t2, 1

sw \$t2, 0(\$t1)

addi \$t0, \$t0, 1

j Loop

EndLoop:

jr \$ra

\$0 - A \$1 - n
\$2 - chave

```

⑤ int busca (int *A, int n, int chave) {
    int i = 0;
    while (i < n) {
        if (A[i] == chave) {
            return i;
        }
        i++;
    }
    return -1;
}

```

busca:

li \$t0, 0 → i

loop:

ori \$t0, \$0, NOTFOUND

slr \$t1, \$t0, 2

add \$t1, \$t1, \$v0

sw \$t2, 0(\$t1)

beq \$t2, \$ra, Found

addi \$t0, \$t0, 1

j loop

Found:

add \$v0, \$t0, \$zero

jr \$ra

NOT FOUND:

li \$v0, -1

jr \$ra

Conversões de MIPS para C

⑥

misterio:

add \$t0, \$a0, \$a1 $t0 = x + y$

srl \$v0, \$t0, 1 $v0 = t0 / 2$

jr \$ra

$\$a0 = x$ $\$a1 = y$

int misterio (int x, int y) {

return (x+y)/2

}

7

func:

```
li $t0, 0 - i = 0
li $v0, 0 - count = 0
```

L:

```
bge $t0, $s1, Fim if (i < n)
sll $t1, $t0, 2 t1 = i * 4
add $t1, $t1, $s0 t1 = &A[i]
lw $t2, 0($t1) t2 = A[i]
slt $t3, $t2, $zero t3 = A[i] < 0
add $v0, $v0, $t3 count += A[i]
addi $t0, $t0, 1 i++
j L
```

Fim:

```
jr $ra
```

\$s0 = &A

\$s1 = n

int func (int *A, int n) {

int i = 0;

int count = 0;

while (i < n) {

if (A[i] < 0) {

count++;

}

i++;

}

return count;

}

8

func:

addi \$t0, \$s1, -1 # t0 = n-1 - i

L:

bltz \$t0, Fim while (i >= 0)

sll \$t1, \$t0, 2 t1 = i * 4

add \$t1, \$t1, \$s0 t1 = &A[i]

lw \$t2, 0(\$t1) t2 = A[i]

add \$t2, \$t2, \$t0 t2 = A[i] + i

sw \$t2, 0(\$t1) t2 = t1

addi \$t0, \$t0, -1 i--

j L

Fim:

jr \$ra

s0 = &A s1 = n

void func (int *A, int n) {

int i = n - 1;

while (i >= 0) {

A[i] = A[i] + i;

i--;

}

}

9

func:

li \$t0, 0 $i = 0$

L:

bge \$t0, \$s2, Fim while ($i < n$)

sll \$t1, \$t0, 2 $t_1 = i * 4$

add \$t3, \$t1, \$s0 $t_3 = \&A[i]$

add \$t4, \$t1, \$s1 $t_4 = \&B[i]$

lw \$t5, 0(\$t3) $t_5 = A[i]$

lw \$t6, 0(\$t4) $t_6 = B[i]$

slt \$t7, \$t5, \$t6 $t_7 = A[i] - B[i]$

beq \$t7, \$zero, Skip if ($A[i] - B[i] == 0$) }

sw \$t6, 0(\$t3) $t_6 = A[i]$ }

Skip:

addi \$t0, \$t0, 1 $i++$

j L

Fim:

jr \$ra

$S_0 = \&A$ $S_1 = \&B$ $S_2 = n$

void func(int *A, int *B, int n)

int i = 0;

while ($i < n$) {

if ($A[i] < B[i]$) {

$A[i] = B[i];$

}

$i++$;

}

}

10

```
func:
    li    $t0, 0    i = 0;
L:
    sll   $t1, $t0, 2    t1 = i * 4
    add   $t1, $t1, $s0    t1 = &A[i]
    lw    $t2, 0($t1)    t2 = A[i]
    beq   $t2, $s2, Achou    if (A[i] == alvo)
    addi  $t0, $t0, 1    i++
    blt   $t0, $s1, L    do while
Achou:
    add   $v0, $t0, $zero    i
    jr    $ra
```

$S_0 = \&A$ $S_1 = n$ $S_2 = alvo$

```
void func (int *A, int n, int alvo)
{
    int i = 0;
    do {
        if (A[i] == alvo) {
            return i;
        }
        i++;
    } while (i < n);
    return i;
}
```

Sem função

C → MIPS

① <code>int temp = a;</code> <code>a = b;</code> <code>b = temp;</code>	<pre>add \$t0, \$v0, \$zero add \$v0, \$v1, \$zero add \$v1, \$t0, \$zero</pre>	$v_0 = a$ $v_1 = b$
② <code>if (x < y) {</code> <code> resultado = x + z;</code> <code>}</code> <code>else {</code> <code> resultado = y - z;</code> <code>}</code>	<pre>slt \$t0, \$v1, \$v0 beq \$t0, \$v1, else add \$s3, \$s0, \$s2 j Fim else: sub \$s3, \$v1, \$v0 Fim:</pre>	$v_0 = x$ $v_1 = y$ $S_2 = z$ $S_3 = \text{resultado}$

③ $home = 0;$
 for ($i = 1; i \leq n; i += 2$)
 $home += i;$
}

$li \$v1, 0$
 $li \$t0, 0$
 Loop:
 $set \$t1, \$0, \$zero$
 $one \$t1, $zero, Fini$
 $add \$v1, $v1, $t0$
 $addi $t0, $t0, 2$
 j Loop
 Fini:

$home = 0$ $v1 = home + 0 = i$

④ $resultaddr = A[i] + A[j];$

$S0 = 2A$ $S1 = i$ $S2 = j$
 $S3 = resultaddr$

$ll $t0, $v1, 2$
 $add $t0, $t0, $t0$
 $lw $t1, 0($t0)$

$ll $t2, $v2, 2$
 $add $t2, $t2, $t2$
 $lw $t3, 0($t2)$

$add $3, $t1, $t3$

⑤ $cnt = 0;$
 $i = 0;$
 while ($i < n$)
 if ($(V[i] > x)$)
 $cnt++;$
 j
 $i++;$
 }

$li $v = 40$
 $n = v1$
 $x = v2$
 $cnt = v3$
 $i = t0$

$li $v3, 0$
 $li $t0, 0$
 Loop:
 bge \$t0, \$v3, EndLoop
 $ll $t1, $t0, 2$
 $add $t1, $t1, $v0$
 $lw $t2, 0($t1)$

$ll $t3, $t2, $v2$
 $one $t2, $zero, Skip$
 $add $v3, $v3, 1$
 Skip:
 $add $t0, $t0, 1$
 j Loop
 EndLoop:

MIPS $\rightarrow$ C

⑥	add \$t0, \$x0, \$x1	$x_0 = c$		$resultado = (a-b) - (x * c);$
	mul \$t1, \$x1, \$x2	$x_1 = b$		
	mul \$x3, \$t0, \$t1	$x_2 = c$		

```
⑦
slt $t0, $s0, $s1    # t0 = (x < y)
beq $t0, $zero, L1    if (t0 == 0)
slt $t1, $s1, $s2    # t1 = (y < z)
beq $t1, $zero, L1    if (t1 == 0)
addi $s3, $zero, 1    # resultado = 1
j L2
L1:
add $s3, $zero, $zero # resultado = 0
L2:
```

$x_0 = x$	$if ((x < y) \&\& (y < z)) \{$	
$x_1 = y$		$resultado = 1;$
$x_2 = z$		$\} else \{$
$x_3 = resultado$		$resultado = 0;$
	$\}$	

8

```

li    $t0, 0    i = 0;
L:
bge   $t0, $s1, Fim    while (i < n)
sll   $t1, $t0, 2    t1 = i * 4
add   $t1, $t1, $s0    t1 = &A[i]
lw    $t2, 0($t1)    t2 = A[i]
add   $t2, $t2, $s2    t2 = t2 + k
sw    $t2, 0($t1)
addi  $t0, $t0, 1    i++
j     L
Fim:

```

```

int i = 0;
while (i < n) {
    A[i] = A[i] + k;
    i++;
}

```

$s_0 = \&A$

$s_1 = n$

$s_2 = k$

9

```
li    $t0, 0    i = 0
```

```
L:
```

```
bge   $t0, $s2, Fim    while (i < n)
```

```
sll   $t1, $t0, 2    t1 = i * 4 ✓
```

```
add   $t2, $t1, $s0    t2 = &A[i] ✓
```

```
lw    $t3, 4($t2)    t3 = A[i+1] ✓
```

```
add   $t4, $t1, $s1    t4 = &B[i]
```

```
lw    $t5, 0($t4)    t5 = B[i]
```

```
add   $t3, $t3, $t5    t3 = t3 + t5
```

```
sw    $t3, 0($t4)
```

```
addi  $t0, $t0, 1    i++
```

```
j     L
```

```
Fim:
```

```
int i = 0;
```

```
while (i < n) {
```

```
    B[i] = A[i+1] + B[i];
```

```
    i++;
```

```
}
```

$v_0 = \&A$

$v_1 = \&B$

$v_2 = n$

15

```
li    $v0, 0 → k
addi  $t0, $s1, -1 → i = n-1
L:
bltz  $t0, Fim  while (i >= 0)
sll   $t1, $t0, 2  t1 = i * 4
add   $t1, $t1, $s0  t1 = &V[i]
lw    $t2, 0($t1)  t2 = V[i]
slt   $t3, $s2, $t2  t3 = lim - V[i]
beq   $t3, $zero, Skip  if (V[i] > lim)
addi  $v0, $v0, 1  k++
Skip:
addi  $t0, $t0, -1  i--
j     L
Fim:
```

```
int k = 0;
int i = n-1;
while (i >= 0) {
    if (V[i] > lim) {
        k++;
    }
    i--;
}
```

s0 := k
s1 := n
s2 := lim